

33. Hardware / Platform Technical Requirements (V1)

33.1 Target Hardware

The MVP shall target a single-device deployment on a Raspberry Pi 5 with an attached Raspberry Pi Touch Display 2 and 64GB microSD storage. The system shall not require any AI accelerator or AI HAT for V1. Raspberry Pi 5 provides a 64-bit quad-core Arm Cortex-A76 at 2.4GHz, and Touch Display 2 supports 720×1280 resolution, up to five simultaneous touch points, and mouse-equivalent touch interactions.

33.2 Required Bill of Materials

Required hardware

- Raspberry Pi 5
- Raspberry Pi Touch Display 2
 - 7-inch strongly recommended for the multi-panel command view
 - 5-inch acceptable only for a tighter footprint
- 64GB microSD card, Class A2 / high-endurance preferred
- Official or equivalent 5V/5A USB-C power supply
- Raspberry Pi 5 case
- Active cooling solution compatible with Pi 5
 - official case with fan or Active Cooler class solution
- Touch Display 2 mounting hardware / stand-offs / enclosure support
- Optional keyboard + mouse for provisioning and maintenance only

Explicitly not required in V1

- AI HAT / AI HAT+
- SSD or NVMe boot
- Pi Zero handheld
- camera, mic, speaker, UPS, second display

Pi 5's official platform guidance recommends proper fan support for Pi 5 cooling, and Touch Display 2 is powered directly by the host Raspberry Pi without a separate display power supply.

34. System Constraints

34.1 Operating System

The system shall run on Raspberry Pi OS (64-bit) with desktop, installed via Raspberry Pi Imager. Raspberry Pi officially provides both 32-bit and 64-bit Raspberry Pi OS editions, and the 64-bit desktop image shall be the baseline for this project.

34.2 Storage

The system shall assume 64GB microSD as the baseline persistent storage target. The implementation shall keep local disk usage below this threshold under normal MVP operation and support later migration to larger media if retention needs grow.

34.3 Power

The system shall assume a 5V/5A USB-C power budget and must not rely on underpowered USB-C supplies. Pi 5 documentation recommends a 5V/5A PSU for full peripheral power availability.

34.4 Touchscreen

The application shall be designed for portrait 720×1280 touch-first interaction using Touch Display 2. It must support:

- single-touch tap
- drag/pan

- vertical scrolling
- touch-based panel interaction
- optional pinch/zoom where the UI framework supports it reliably

Touch Display 2 supports multitouch and mouse-equivalent desktop control, but the app should not depend on complex multitouch gestures for core workflows.

34.5 Networking

The system shall assume always-on Wi-Fi access for feed polling, map tile loading, and dashboard refresh. Pi 5 has built-in dual-band wireless networking suitable for this use.

35. Platform Functional Requirements

35.1 Boot and Runtime

- The system shall boot directly into Raspberry Pi OS desktop.
- The dashboard application shall auto-start on boot without manual terminal steps.
- The device shall be usable as an always-on kiosk/workstation.
- A power loss followed by restore shall return the device to the dashboard automatically.

35.2 Display and Layout

- The dashboard shall be optimized for portrait 720×1280.
- The default screen shall be a multi-panel command view consisting of:
 - central U.S. map
 - left-side filter/layer rail

- right-side priority/detail rail
- bottom or lower-third feed/timeline region
- The UI shall remain legible at arm's-length desk viewing distance on the Touch Display 2.

35.3 Touch Interaction

The app shall support full analyst workflow via touchscreen, including:

- selecting event markers
- opening/closing detail panes
- toggling heatmap and event layers
- filtering by time, type, state, confidence
- scrolling event feed
- drilling into hotspot detail
- zooming/panning the map

A keyboard/mouse may be used for maintenance only and shall not be required for normal operation. Touch Display 2 supports mouse-equivalent control, so the app should be implementable as a touch-friendly desktop web app or local kiosk app.

35.4 Local Processing

All V1 data ingestion, normalization, clustering, scoring, and rendering orchestration shall run on the Pi 5 CPU with no hardware AI acceleration. The application shall not assume CUDA, GPU inference, or Hailo/NPU availability.

35.5 Storage and Data Retention

- The system shall persist:

- normalized event records
 - hotspot/cluster snapshots
 - source metadata
 - logs
 - cached UI state where needed
- MVP retention shall be configurable
- Default retention target: 30–90 days of event data on 64GB storage
- The system shall degrade gracefully if retention is shortened to preserve space

35.6 Update Scheduling

- The backend shall support scheduled ingestion every 30 minutes by default
- The scoring pipeline shall recompute hotspot state after each ingest cycle
- The UI shall display:
 - last ingestion timestamp
 - last successful update timestamp
 - data freshness per event/hotspot where available

35.7 Offline / Failure Behavior

- If feed polling fails, the dashboard shall continue to show last known data
- The UI shall display a stale-data indicator when freshness exceeds configured thresholds
- System logs shall persist ingestion failures locally

- The app shall not crash or become unusable on temporary network loss

36. Technical Implementation Requirements for Coding Agents

36.1 Application Architecture

The implementation shall be split into these components:

1. Ingestion service
 - polls configured feeds/APIs
 - normalizes event records
 - writes to local database
2. Scoring / clustering service
 - computes event clusters
 - computes hotspot intensity
 - computes trend state and confidence
3. API layer
 - serves normalized events
 - serves hotspot summaries
 - serves dashboard metadata/config
 - exposes health/readiness endpoints
4. Frontend workstation
 - touch-first dashboard UI

- map, feed, filters, detail panes
- kiosk-friendly runtime

36.2 Local Database

The MVP shall use a local database suitable for Pi 5 deployment:

- SQLite preferred for MVP simplicity
- schema must support:
 - events
 - sources
 - hotspots
 - clusters
 - ingest_runs
 - app_settings

36.3 Frontend Delivery

The frontend shall be deployable in one of these modes:

- local web app opened in Chromium kiosk mode
- local desktop app shell if justified

For V1, the preferred approach is:

- local web app

- auto-launched in Chromium kiosk/fullscreen

Raspberry Pi OS desktop includes mainstream browser support in official desktop images, which is suitable for kiosk-style local delivery.

36.4 Kiosk Startup

The system shall support:

- auto-login into desktop session
- automatic application launch on boot
- fullscreen kiosk mode
- screen blanking disabled during active use

36.5 Performance Targets

On Pi 5 hardware, the system should meet these V1 targets:

- cold boot to usable dashboard: under 90 seconds
- dashboard initial load after app launch: under 8 seconds
- map interaction latency: under 250 ms for pan/select in normal state
- event detail panel open: under 500 ms
- 30-minute ingest cycle completion: under 5 minutes under normal feed volume
- no thermal shutdown or UI freeze during normal operation

These are product targets, not guaranteed hardware benchmarks.

37. Software Environment Requirements

37.1 Baseline Software Stack

Coding agents shall assume:

- Raspberry Pi OS (64-bit) with desktop
- Python 3.x available
- Node.js optional if frontend stack requires it
- Chromium available for kiosk rendering
- systemd available for service management

37.2 Service Management

All long-running backend components shall be installable as systemd services:

- ingestion service
- scoring service
- API service
- kiosk launcher or app bootstrap service if needed

37.3 Logging

The system shall log to local disk with rotation for:

- ingestion runs
- API errors
- scoring errors

- UI launch failures
 - startup/health checks
-

38. Recommended Physical Form Requirements

38.1 Desk Deployment

The final MVP shall function as a desk-mounted or stand-mounted portrait workstation in a dorm room.

38.2 Cooling and Stability

Because Pi 5 is a higher-performance board, the build shall include active cooling and an enclosure that allows continuous always-on operation. Raspberry Pi's official hardware documentation recommends official fan solutions for Pi 5.

38.3 Display Connection

Touch Display 2 shall be connected via its intended Raspberry Pi display interface and powered directly by the Pi. Touch Display 2 is designed for plug-and-play integration and direct host power.

39. Non-Functional Requirements

- The system shall be installable by one user on one Pi 5 without cloud infrastructure dependencies for core operation
 - The system shall prefer local persistence and local serving over remote hosting
 - The system shall remain usable after reboot without manual reconfiguration
 - The system shall be maintainable through SSH or local desktop access
 - The system shall not require AI HAT-specific libraries, toolchains, or inference runtimes
-

40. Acceptance Criteria — Platform Layer

The platform layer is complete when all of the following are true:

1. Pi 5 boots into Raspberry Pi OS desktop and auto-launches the dashboard
 2. Touch Display 2 works for full core interaction without keyboard/mouse
 3. The system runs continuously for 24 hours without thermal or power instability
 4. Ingestion runs automatically on schedule
 5. Data persists across reboots
 6. The UI remains usable and legible in portrait touchscreen mode
 7. The full app runs with no AI HAT installed
 8. Storage usage remains within the 64GB baseline during normal MVP operation
-

41. Notes for Claude Code / Codex

Assume the hardware target is:

```
hardware_target:  
  board: Raspberry Pi 5  
  ai_hat: false  
  display: Raspberry Pi Touch Display 2  
  display_orientation: portrait  
  resolution: 720x1280  
  touch_input: true  
  storage: 64GB microSD  
  power: 5V_5A_USB_C  
  cooling: active  
  os: Raspberry Pi OS 64-bit with desktop  
  deployment_mode: local_kiosk_workstation
```

Assume the app must be built to:

- run locally on Pi 5
- start automatically at boot
- support touch-first interaction
- store data locally
- update on a 30-minute schedule
- avoid any dependency on AI accelerator hardware

ENGINEERING SPEC — FLASHPOINT V1

Raspberry Pi 5 Local Kiosk Workstation

U.S. Flashpoints Monitor

0. PURPOSE

Build a near-real-time U.S. flashpoints monitoring workstation that runs entirely on a single Raspberry Pi 5 with a touchscreen, local storage, and no AI HAT.

The application must:

- ingest public unrest-related event data on a schedule

- normalize and store events locally
- compute hotspot/cluster state
- expose a local API
- render a touch-first multi-panel dashboard in fullscreen kiosk mode
- run reliably as an always-on dorm workstation

This spec is optimized for implementation by coding agents.

1. HARDWARE TARGET

1.1 Required Hardware

- Raspberry Pi 5
- Raspberry Pi Touch Display 2
- 64GB microSD card
- 5V/5A USB-C power supply

- Pi 5 case
- active cooling
- Wi-Fi connectivity

1.2 Optional Setup Hardware

- USB keyboard
- USB mouse
- external monitor for provisioning only

1.3 Explicit Non-Requirements

- no AI HAT
- no Pi Zero handheld in V1
- no SSD/NVMe required
- no camera
- no microphone
- no speaker
- no battery backup required in V1

1.4 Runtime Assumptions

- single-user device
- always-on
- desk-mounted portrait display
- local browser kiosk frontend
- local database
- local services managed by systemd

2. ENVIRONMENT

2.1 OS

- Raspberry Pi OS (64-bit) with desktop

2.2 Display

- orientation: portrait
- target resolution: 720x1280
- touch-first interaction required

2.3 Runtime Stack

Preferred baseline:

- Python 3.11+
- FastAPI for backend API
- SQLite for local database
- APScheduler or cron/systemd timers for scheduled ingest
- a modern frontend stack:
 - preferred: React + Vite
 - acceptable: Next.js in static/local mode
 - acceptable for MVP: plain JS app
- Leaflet or MapLibre GL JS for map rendering
- Chromium for kiosk mode
- systemd for service management

3. SYSTEM GOALS

3.1 Core User Goal

The user should be able to open the dashboard and quickly determine:

- where unrest is happening in the U.S.
- why those areas matter
- what specific events drive the hotspots
- whether activity is rising, stable, or cooling

3.2 MVP Product Shape

The MVP is:

- map-first
- event-driven
- hotspot-aware

- touch-first
- near-real-time
- source-backed
- local-first

The MVP is not:

- a predictive engine
- a chat assistant
- a collaboration platform
- a mobile app
- a military sim
- a generic news dashboard

4. ARCHITECTURE

4.1 High-Level Components

The system shall contain 4 main components:

1. ingestion service
2. scoring/clustering service
3. local API service
4. frontend kiosk application

4.2 Architecture Diagram

[External Feeds / APIs]

|

v

[Ingestion Service]

|

v

[Normalization / Deduplication]

|

v

[SQLite Database]

|

+-----+

|

|

v

v

[Scoring / Clustering] [Local API]

|

|

+-----+-----+

|

v

[Touchscreen Frontend]

|

v

[Chromium Kiosk Mode]

5. REPOSITORY STRUCTURE

Use this structure:

flashpoint/

README.md

.env.example

requirements.txt

pyproject.toml

scripts/

install.sh

setup_kiosk.sh

setup_services.sh

rotate_logs.sh

backend/

app/

main.py

config.py

database.py

models.py

schemas.py

utils/

routes/

health.py

events.py

hotspots.py

trends.py

filters.py

services/

ingestion/

base.py

gdelt.py

acled.py

parser.py

normalizer.py

deduper.py

scoring/

cluster.py

hotspot.py

confidence.py

trends.py

jobs/

scheduler.py

ingest_job.py

score_job.py

storage/

migrations/

seed/

logs/

frontend/

package.json

vite.config.ts

src/

main.tsx

App.tsx

styles/

components/

layout/

map/

feed/

detail/

filters/

priorities/

timeline/

pages/

hooks/

services/

api.ts

types/

deploy/

systemd/

flashpoint-api.service

flashpoint-jobs.service

flashpoint-kiosk.service

chromium/

kiosk.sh

docs/

PRD.md

ENGINEERING_SPEC.md

API.md

SCHEMA.md

6. BOOT FLOW

6.1 Desired Boot Behavior

On power-on:

1. Pi boots into Raspberry Pi OS desktop
2. user auto-login occurs
3. backend API service starts
4. ingestion/scoring scheduler starts
5. Chromium launches in fullscreen kiosk mode
6. dashboard loads local frontend
7. frontend fetches current hotspot/event state from local API

6.2 Failure Expectations

- if frontend fails, Chromium should restart
- if API fails, systemd should restart it
- if ingestion fails, previous data remains visible

- if network is unavailable, dashboard still loads last-known state

7. INSTALLATION REQUIREMENTS

7.1 One-Command Install

Provide an install script that:

- installs apt dependencies
- installs Python dependencies
- installs frontend dependencies
- builds frontend
- initializes database
- installs systemd units
- configures Chromium kiosk launcher
- sets touchscreen orientation if needed

Preferred command:

```
bash scripts/install.sh
```

7.2 Required apt Dependencies

At minimum:

- python3

- python3-venv

- python3-pip

- sqlite3

- chromium-browser or chromium

- unclutter

- xdotool

- git

- curl

- jq

- build-essential

- nodejs

- npm

7.3 Python Dependencies

At minimum:

- fastapi

- uvicorn

- pydantic

- sqlalchemy

- aiosqlite or sqlite support

- apscheduler

- requests and/or httpx

- python-dateutil

- pandas optional

- scikit-learn optional for clustering

- geopy optional if geocoding is needed

- python-dotenv

7.4 Frontend Dependencies

At minimum:

- react
- react-dom
- vite
- leaflet or maplibre-gl
- a lightweight state library optional
- charting library optional for timeline panel

8. DATABASE REQUIREMENTS

8.1 Database Engine

- SQLite for MVP
- file path default:

/home/pi/flashpoint/data/flashpoint.db

8.2 Required Tables

events

Fields:

- id TEXT PRIMARY KEY

- external_id TEXT

- source_id TEXT NOT NULL

- title TEXT

- summary TEXT

- event_type TEXT NOT NULL

- city TEXT

- state TEXT

- country TEXT DEFAULT 'US'

- latitude REAL

- longitude REAL

- occurred_at DATETIME NOT NULL
- ingested_at DATETIME NOT NULL
- source_url TEXT
- source_name TEXT
- source_count INTEGER DEFAULT 1
- confidence_score REAL DEFAULT 0.0
- severity_score REAL DEFAULT 0.0
- cluster_id TEXT
- trend_state TEXT
- is_active INTEGER DEFAULT 1
- raw_payload_json TEXT

Indexes:

- idx_events_occurred_at
- idx_events_state
- idx_events_event_type
- idx_events_cluster_id

- idx_events_lat_lon

sources

Fields:

- id TEXT PRIMARY KEY

- name TEXT NOT NULL

- source_type TEXT NOT NULL

- trust_weight REAL DEFAULT 0.5

- base_url TEXT

- is_enabled INTEGER DEFAULT 1

- last_fetch_at DATETIME

- last_success_at DATETIME

- last_error TEXT

ingest_runs

Fields:

- id TEXT PRIMARY KEY

- source_id TEXT
- started_at DATETIME NOT NULL
- completed_at DATETIME
- status TEXT NOT NULL
- records_fetched INTEGER DEFAULT 0
- records_inserted INTEGER DEFAULT 0
- records_updated INTEGER DEFAULT 0
- records_skipped INTEGER DEFAULT 0
- error_message TEXT

hotspots

Fields:

- id TEXT PRIMARY KEY
- name TEXT
- centroid_lat REAL
- centroid_lon REAL
- bounding_box_json TEXT

- event_count INTEGER DEFAULT 0
- confidence_score REAL DEFAULT 0.0
- severity_score REAL DEFAULT 0.0
- momentum_score REAL DEFAULT 0.0
- spread_score REAL DEFAULT 0.0
- strategic_score REAL DEFAULT 0.0
- priority_score REAL DEFAULT 0.0
- trend_state TEXT
- status_label TEXT
- last_computed_at DATETIME NOT NULL

hotspot_events

Fields:

- hotspot_id TEXT NOT NULL
- event_id TEXT NOT NULL
- PRIMARY KEY (hotspot_id, event_id)

clusters

Fields:

- id TEXT PRIMARY KEY
- centroid_lat REAL
- centroid_lon REAL
- event_count INTEGER DEFAULT 0
- started_at DATETIME
- last_updated_at DATETIME
- trend_state TEXT
- cluster_score REAL DEFAULT 0.0

app_settings

Fields:

- key TEXT PRIMARY KEY
- value TEXT NOT NULL
- updated_at DATETIME NOT NULL

system_logs

Fields:

- id INTEGER PRIMARY KEY AUTOINCREMENT

- level TEXT NOT NULL

- component TEXT NOT NULL

- message TEXT NOT NULL

- created_at DATETIME NOT NULL

- metadata_json TEXT

9. DATA MODEL RULES

9.1 Event Rules

- every event must have:

- source

- event_type

- occurred_at
- location or best-known regional location
- events missing lat/lon may still be stored if state-level placement is available
- unresolved events may be flagged low-confidence

9.2 Deduplication Rules

Events should be deduplicated by a combination of:

- source identity
- title similarity
- time proximity
- location proximity
- event type similarity

Agent implementation may use a heuristic score.

If score exceeds threshold, merge/update existing event instead of inserting new one.

9.3 Hotspot Rules

A hotspot is a computed object, not a raw ingest object.

A hotspot should consider:

- event count
- recency
- severity
- source corroboration
- geographic concentration
- spread across nearby locations

10. INGESTION REQUIREMENTS

10.1 Ingestion Sources

Backend must be source-pluggable.

Implement source adapters via a common interface.

Example base interface:

```
class SourceAdapter:
```

```
    def fetch(self, since: datetime | None) -> list[dict]:
```

```
        ...
```

```
    def normalize(self, raw_record: dict) -> dict:
```

```
        ...
```

10.2 Ingestion Schedule

- default poll interval: every 30 minutes

- configurable via environment variable

10.3 Ingestion Job Responsibilities

Each run must:

1. fetch new raw records

2. normalize to event schema
3. deduplicate
4. insert/update database
5. record ingest run stats
6. trigger hotspot recomputation

10.4 Ingestion Robustness

- failed source fetch must not break other sources
- partial ingest failure must be logged
- stale data must remain visible
- retries optional in MVP but desirable

11. SCORING REQUIREMENTS

11.1 Severity Score

Use a normalized 0.0–1.0 score.

Inputs may include:

- event type
- violence-related tags
- disruption-related tags
- corroboration strength

11.2 Confidence Score

Use a normalized 0.0–1.0 score.

Inputs:

- number of corroborating sources
- source trust weights
- completeness of event fields
- location precision
- recency

11.3 Momentum Score

Use a normalized 0.0–1.0 score.

Inputs:

- recent increase in event count
- recent increase in severe event count
- clustering over short time windows

11.4 Priority Score

Weighted combination:

priority_score =

severity_score * w1 +

momentum_score * w2 +

confidence_score * w3 +

spread_score * w4 +

strategic_score * w5

Default weight order:

- severity highest

- momentum second

- confidence third

- spread fourth

- strategic fifth

Store weights in config.

11.5 Trend State

Trend state must be one of:

- rising

- stable

- cooling

Trend is computed at hotspot and cluster level.

12. CLUSTERING REQUIREMENTS

12.1 Geographic Clustering

Implement basic clustering for events using:

- DBSCAN preferred
- or simple geospatial radius grouping for MVP

12.2 Time Window

Clusters should favor recent time windows.

Default:

- primary operational window: last 24 hours
- high-signal priority window: last 6 hours

12.3 Output

Each event may belong to zero or one active cluster in MVP.

Each cluster should store:

- centroid
- count
- last update
- score
- trend

13. API REQUIREMENTS

13.1 API Style

- local REST API
- JSON responses
- no auth required in single-device MVP
- route prefix: /api/v1

13.2 Required Endpoints

GET /api/v1/health

Returns:

- app status
- db status
- last ingest time
- uptime

GET /api/v1/events

Query params:

- start_time
- end_time
- state
- event_type
- min_confidence
- limit
- bbox optional

GET /api/v1/events/{event_id}

Returns full event detail.

GET /api/v1/hotspots

Returns current hotspot list sorted by priority.

GET /api/v1/hotspots/{hotspot_id}

Returns hotspot detail plus associated events.

GET /api/v1/priorities

Returns top 3 most pressing areas/events.

GET /api/v1/trends

Returns active trend summaries.

GET /api/v1/filters

Returns available filter metadata.

GET /api/v1/system/status

Returns freshness, ingest state, storage stats.

13.3 Response Performance

- list endpoints should respond in under 1 second on warm cache
- hotspot and priority endpoints should be optimized for immediate dashboard load

14. FRONTEND REQUIREMENTS

14.1 Layout

Portrait touch-first multi-panel UI:

- center: U.S. map
- left rail: filters/layers

- right rail: priorities + detail

- lower panel: feed/timeline

14.2 Required Views

Main Command View

Must show:

- map

- event markers

- heatmap toggle

- priority panel

- recent feed

- last updated indicator

Event Detail View

Shown in right rail or modal panel.

Must show:

- title
- location
- event type
- time
- summary
- confidence
- source list/count
- related nearby events

Hotspot Detail View

Must show:

- hotspot name or region
- event count
- trend
- confidence
- why-it-matters summary
- list of events in hotspot

14.3 Touch Requirements

All primary features must work with touch only:

- tap
- scroll
- drag
- map pan
- map zoom
- filter toggles
- open/close details

No hover-only interactions.

14.4 Styling Requirements

- dark theme
- high contrast
- tactical but restrained

- clean typography
- subtle animated glow/pulse allowed
- no fake clutter
- no noisy HUD chrome

15. KIOSK REQUIREMENTS

15.1 Chromium Launch

The app must launch in Chromium kiosk mode.

Preferred launch flags:

- --kiosk
- --incognito optional
- --disable-infobars
- --noerrdialogs

- --disable-session-crashed-bubble

15.2 Kiosk URL

Serve frontend locally, for example:

<http://localhost:3000>

or

<http://127.0.0.1:8000/app>

15.3 Cursor / Idle

- hide mouse cursor when inactive

- do not allow sleep/screen blank during active runtime

16. SYSTEMD REQUIREMENTS

Provide these unit files:

16.1 flashpoint-api.service

Runs FastAPI/Uvicorn backend.

16.2 flashpoint-jobs.service

Runs scheduler for ingest + scoring loop.

16.3 flashpoint-kiosk.service

Starts Chromium kiosk after graphical session is ready.

16.4 Restart Policy

All services:

- Restart=always or on-failure as appropriate
- short restart delay
- logs to journalctl

17. CONFIGURATION REQUIREMENTS

17.1 Environment File

Provide .env.example with keys such as:

APP_ENV=production

APP_HOST=127.0.0.1

APP_PORT=8000

DB_PATH=/home/pi/flashpoint/data/flashpoint.db

INGEST_INTERVAL_MINUTES=30

HOTSPOT_WINDOW_HOURS=24

PRIORITY_WINDOW_HOURS=6

RETENTION_DAYS=60

FRONTEND_URL=http://localhost:3000

KIOSK_ENABLED=true

DEFAULT_MAP_CENTER_LAT=39.5

DEFAULT_MAP_CENTER_LON=-98.35

DEFAULT_MAP_ZOOM=3

17.2 Runtime Config

The system must allow adjustment of:

- ingestion interval
- priority weights
- retention days
- hotspot thresholds
- trend thresholds

18. LOGGING AND OBSERVABILITY

18.1 Required Logs

Log:

- startup
- ingest run started/completed
- source fetch failures
- dedupe stats
- hotspot recomputation
- API errors
- kiosk launch failures

18.2 Log Storage

Store log files locally with rotation.

Also allow inspection via journalctl.

18.3 Health Signals

Expose:

- last successful ingest
- number of active events
- number of hotspots

- database size

- free disk estimate

19. STORAGE MANAGEMENT

19.1 Retention

Default retention:

- 60 days of events

- configurable

19.2 Cleanup Job

Provide periodic cleanup job:

- prune events older than retention unless flagged

- vacuum/analyze SQLite optional

- remove stale logs over threshold

19.3 Capacity Guardrails

App should warn if:

- disk utilization exceeds threshold
- DB file grows faster than expected
- retention needs reduction

20. ACCEPTANCE CRITERIA

20.1 Platform Acceptance

- device boots successfully into kiosk dashboard
- touch interaction works throughout core flows
- backend services auto-start
- ingest runs every 30 minutes
- data persists across reboot

- app runs with no AI HAT installed

20.2 MVP Product Acceptance

- dashboard shows U.S. map and recent event markers
- heatmap can be toggled
- top 3 priorities render correctly
- tapping a hotspot or event opens details
- trend labels render
- stale-data state is visible if ingest fails

20.3 Stability Acceptance

- survives 24-hour run without manual intervention
- no thermal shutdowns
- no repeated frontend crash loop
- no database corruption on reboot

21. IMPLEMENTATION PRIORITY

Phase 1

- backend scaffolding
- database schema
- health route
- basic ingest pipeline
- sample seeded data

Phase 2

- hotspot scoring
- cluster logic
- API endpoints

Phase 3

- frontend command view

- event markers

- detail panel

- priorities pane

Phase 4

- heatmap

- trend states

- kiosk boot flow

- systemd packaging

Phase 5

- polish

- cleanup jobs

- log rotation

- error handling

22. CODING AGENT DIRECTIVES

When implementing:

- optimize for reliability over visual novelty
- prefer local simplicity
- avoid unnecessary cloud dependencies
- avoid AI-specific dependencies
- keep data model explicit
- keep services separable
- make install reproducible
- make kiosk startup deterministic

Do not:

- add authentication in MVP
- add chat features
- add mobile-specific flows

- assume GPU/NPU
- block the build on unavailable feeds
- overengineer infrastructure

23. HARDWARE TARGET SUMMARY

Use this exact target:

hardware_target:

board: "Raspberry Pi 5"

display: "Raspberry Pi Touch Display 2"

display_orientation: "portrait"

resolution: "720x1280"

touch_input: true

storage: "64GB microSD"

ai_hat_required: false

power: "5V/5A USB-C"

cooling: "active"

os: "Raspberry Pi OS 64-bit with desktop"

deployment_mode: "local fullscreen kiosk workstation"

24. DEFINITION OF DONE

This platform layer is done when a fresh Pi 5 can be provisioned, rebooted, and left on a desk, and it will:

- boot into the dashboard automatically
- ingest and store data on schedule
- compute hotspots locally
- render a usable touch-first command interface
- continue working without any AI HAT or secondary device

